Criteria	Max Points	Excellent (100%)
Code Readability & Organization	10	Single-responsibility functions, consistent naming, logical cell sequencing. Markdown docs sufficient for onboarding without walkthrough.
Code Reusability & Modularity	10	Well-timed abstractions. Shared utilities (API wrappers, parsers, agent patterns) factored out. Zero copy-paste across components.
Error Handling & Failure Recovery		Closed-loop recovery: detect → diagnose → select strategy (backoff, fallback model, cache, degrade gracefully) → log outcome. Covers API limits, malformed docs, LLM refusals.
Document Processing & Text Extraction	10	Full lifecycle: ingest, clean, chunk, tag metadata. Chunk size/overlap tuned for retrieval quality. Handles malformed/empty docs without silent failure.
Named Entity Recognition & Domain Extraction		Domain-specific entities beyond generic NER (obligation clauses, drug interactions, regulatory refs, price signals). Typed, deduplicated, linked to source spans.
Vector Store & Retrieval Grounding	10	Appropriate embedding model + similarity metric. Metadata-filtered search. Validated top-k relevance. LLM claims traceable to source chunks — no ungrounded assertions.
Agent Building & Integration	10	Clear mandate per agent, Pydantic output contracts, domain-expert prompts. Decomposes complex queries into sub-tasks. Tool selection intent-driven, not hardcoded. Memory strategy manages context window deliberately. Confidence quantified and propagated.
Multi-Agent Orchestration	10	Agents composed, not just sequenced. Coordination pattern (parallel, consensus, delegation) fits the problem. Conflicts reconciled with defensible strategy. Per-agent token/cost attribution tracked.
Workflow & State Management	10	Conditional routing on content/risk/confidence. Human-in-the-loop where stakes warrant. Fallback paths for degraded operation. State transitions explicit and auditable.
Observability & Cost-Latency Design	10	End-to-end Langfuse traces with span hierarchy. Every LLM call attributed to agent + step. Custom quality scores. Cost-latency tradeoffs deliberate: model tiering (GPT-4 vs mini by task), caching policy, batch strategy — with documented rationale.
Accuracy & Trustworthiness	10	Zero hallucinated facts. Mitigation by design: grounding enforced, schemas constrain generation, confidence gates low-certainty outputs. Eval dataset (golden pairs / annotated examples) validates quality — not just spot-checks. Withstands SME review.
Deployment & Scalability	5	FastAPI endpoints tested. Config externalized (dotenv/secrets). Batch + cache reduce redundant LLM calls. Handles realistic workloads.
UI & Usability	5	Gradio interface with file upload, tabbed results, embedded visualizations. Non-technical stakeholder can use without tutorial.
Unique Enhancements / Additional Features (Bonus 10 pts)	5*	Substantive feature beyond requirements: knowledge graphs, predictive analytics, custom eval framework, novel coordination pattern, or domain innovation showing independent judgment.

Good (90%)	Satisfactory (75%)	Needs Improvement (40%)
Mostly clean with minor naming/structure inconsistencies.	Readable but inconsistent. Minimal documentation.	Unstructured, no comments, hard to follow.
Mostly reusable, minor redundancy.	Some reuse, noticeable duplication.	Repeated code throughout, no abstraction.
Retry logic present but open-loop (same strategy repeated). Logging lacks diagnostic context.	Some try-except, no recovery loops. Failures crash or return empty.	Minimal handling, frequent crashes.
Parses correctly, chunking works but untuned.	Basic parsing, fixed chunks, no overlap, fails on edge cases.	Parsing breaks frequently. No chunking.
Standard NER works. Some domain extraction, missing key types.	Generic NER only (person/org/date). No domain specificity.	NER absent or mostly incorrect.
Search mostly relevant. Responses reference context but grounding not enforced systematically.	Basic setup, inconsistent search. LLM sometimes asserts beyond retrieved context.	Misconfigured or non-functional. Generation ungrounded.
Good prompts and output structure. Some planning/routing but not systematic. Memory present, unoptimized.	Generic prompts, hardcoded tools, no decomposition, no memory, no confidence scoring.	Agents incomplete or unreliable.
Coordination works. Consensus present but weighting unjustified.	Agents independent, no real coordination. Simple averaging.	Non-functional or unresolved contradictions.
Main-path routing works. Limited HITL. Basic retries.	Linear pipeline, no routing, no HITL, no fallbacks.	No workflow structure. Ad-hoc agent calls.
Tracing works. Cost tracked but model selection uniform — no deliberate tradeoff.	Basic traces, no scoring, single model everywhere, no cost rationale.	No observability. Black-box agents. No cost awareness.
Some guardrails (grounding), no eval dataset. Occasional inaccuracies.	Manual testing only. No systematic mitigation. Some inaccuracies.	Frequent hallucinations, no grounding, no evaluation.
Endpoints work. Mostly clean config. Gaps in caching/batch.	Not production-ready. Hardcoded config.	Notebook-only. Manual cell execution.
Functional UI, some views/visualizations missing.	Basic single-view interface. No visualizations.	Confusing or missing UI.
Meaningfully enhanced existing feature with measurable improvement.	Minor polish — UI tweaks, formatting.	No enhancements.